

Betriebssysteme, Übungsblatt 3, Winter 2025

Aufgabe 1: Datensicherung und -archivierung mit der Hilfe von ZFS

Das Ziel dieser Aufgabe ist die Implementierung eines automatischen Sicherungssystems, das die speziellen Eigenschaften des ZFS-Dateisystems nutzt, um konsistente Backups zu erzeugen. Um möglichst konsistente Zustände zu erreichen, nutzt die Implementierung ZFS-Snapshots. Ein ZFS-Snapshot basiert auf der Copy-on-Write-Technik (CoW) von ZFS. Vorteil der CoW-Technik ist, dass bestehende Datenblöcke niemals überschrieben werden. Wird ein Block geändert, schreibt ZFS die neue Version an eine freie Stelle auf dem Datenträger und aktualisiert anschließend die Verweise. Das Original bleibt unverändert erhalten, bis die Aktualisierung vollständig abgeschlossen ist. Zur Archivierung wird die Funktion `zfs send` verwendet. Diese serialisiert den Snapshot in einen Datenstrom, der in einer Datei an einem anderen Ort (Ordner Y) gespeichert wird. Dies stellt sicher, dass das Backup exakt dem Zustand des Dateisystems zum Zeitpunkt des Snapshots entspricht.

Die Umsetzung erfolgt in einer virtuellen Maschine unter Ubuntu Linux. Zur Vorbereitung wurden im Terminal folgende Schritte durchgeführt.

Erstellung eines ZFS-Pools: Ein Speicherpool namens `pool1` wurde auf einer dedizierten Festplatte (`/dev/sdb`) angelegt: `sudo zpool create pool1 /dev/sdb`

Erstellung des Quell-Datasets (Ordner X): `sudo zfs create pool1/daten_x`

Konfiguration des Zielorts (Ordner Y): Ein lokaler Ordner `~/zfs_backups` dient als Archiv für die `.zfs`-Dateien

Die Lösung wurde in Python implementiert. Um Betriebssystembefehle aus dem Script starten zu können, wurde das Modul `subprocess` verwendet. Zu Beginn des Programms werden die Parameter für das Quell-Dataset (X), der Zielordner (Y) und die maximale Anzahl der Backups definiert. Der Kern der Implementierung ist die Nutzung der Copy-on-Write Funktionalität von ZFS. Bevor Daten übertragen werden, wird ein Snapshot des Datasets erstellt. Dazu wird ein Zeitstempel generiert, um Eindeutigkeit sicherzustellen. Anschließend wird der Befehl `zfs snapshot pool1/daten_x@<zeitstempel>` ausgeführt. Damit werden die Zeiger auf die aktuellen Datenblöcke eingefroren und ein konsistentes Abbild von X sichergestellt, auch wenn währenddessen Schreibzugriffe stattfinden. Dabei werden alle im Dataset enthaltenen Dateien und Unterverzeichnisse implizit rekursiv miterfasst, wodurch die Anforderung einer vollständigen Sicherung von Ordner X erfüllt wird. Anschließend wird mit dem Befehl `zfs send <snapshot>` ein kontinuierlicher Datenstrom, der das gesamte Dataset repräsentiert, generiert. Dieser Datenstrom wird in eine `.zfs`-Datei im Zielordner Y geschrieben. Um die Anzahl der gleichzeitigen Backups zu begrenzen, wird die Anzahl an Backups im Zielordner Y gezählt. Wird die maximale Anzahl erreicht, wird das älteste Backup entfernt. Dazu wird eine alphabetisch geordnete Liste erstellt, von der die älteste erste Datei gelöscht wird. Da die Dateinamen sich nur im Zeitstempel ändern, entspricht die alphabetische auch der chronologischen Reihenfolge. Die Screenshots zeigen das korrekte Verhalten. Zunächst werden drei neue Backups in Y gespeichert. Da danach die maximale Anzahl erreicht ist, wird im Folgenden die älteste Datei gelöscht. Wird das Skript mit dem Argument `restore` aufgerufen, wird die neueste `.zfs`-Datei aus Ordner Y ausgelesen und mittels `sudo zfs receive pool1/daten_x_restore <Datei in ein neues Dataset im Pool wiederhergestellt`.

```

matthias@matthias-VirtualBox:~$ python3 backup.py

--- Starte Backup: 20260227_113917 ---
Erstelle Snapshot: pool1/daten_x@20260227_113917
Sende Snapshot in Datei: /home/matthias/zfs_backups/backup_20260227_113917.zfs
--- Backup abgeschlossen ---

matthias@matthias-VirtualBox:~$ python3 backup.py

--- Starte Backup: 20260227_113950 ---
Erstelle Snapshot: pool1/daten_x@20260227_113950
Sende Snapshot in Datei: /home/matthias/zfs_backups/backup_20260227_113950.zfs
--- Backup abgeschlossen ---

matthias@matthias-VirtualBox:~$ python3 backup.py

--- Starte Backup: 20260227_113953 ---
Erstelle Snapshot: pool1/daten_x@20260227_113953
Sende Snapshot in Datei: /home/matthias/zfs_backups/backup_20260227_113953.zfs
--- Backup abgeschlossen ---

matthias@matthias-VirtualBox:~$ python3 backup.py

--- Starte Backup: 20260227_113955 ---
Erstelle Snapshot: pool1/daten_x@20260227_113955
Sende Snapshot in Datei: /home/matthias/zfs_backups/backup_20260227_113955.zfs
Retention: Lösche altes Backup backup_20260227_113917.zfs
--- Backup abgeschlossen ---

```

Aufgabe 2: Datei-Inkonsistenzen trotz ZFS-Snapshots

ZFS-Snapshots garantieren Crash-Consistency. Das bedeutet, dass das Dateisystem zu jedem Zeitpunkt in einem konsistenten Zustand gesichert wird, vergleichbar mit dem Zustand nach einem plötzlichen Stromausfall. Das Problem ist, dass viele Anwendungen Schreibvorgänge in mehreren Schritten durchführen. Findet ein Snapshot genau zwischen zwei zusammengehörigen Schreibvorgängen statt, ist die Datei im Backup aus Sicht der Anwendung unvollständig, obwohl das Dateisystem selbst fehlerfrei ist.

Um diesen Effekt zu zeigen, wurde ein Experiment mit zwei parallellaufenden Prozessen durchgeführt. Dazu wurde zunächst das Python-Skript `writer.py` gestartet, das zunächst in eine `.txt` Datei schreibt, dann 20 Sekunden wartet und erst danach die nächste Zeile startet. Während dieser Pause wird das Sicherungsscript aus Aufgabe 1 gestartet, das den ZFS-Snapshot erstellt.

Wie im Screenshot zu sehen ist, meldet die Anwendung „Ende geschrieben. Vorgang abgeschlossen“. Betrachtet man jedoch den Inhalt der Datei innerhalb des Snapshots, zeigt sich, dass dort lediglich die erste Zeile enthalten ist (Transaktion Start), die zweite Zeile jedoch nicht (Transaktion Ende). Somit entsteht aus Sicht der Anwendung (z.B. Datenbanken) ein unfertiger Zustand. Beispielsweise könnte Transaktion Start dafür stehen, dass von Konto A 100 Euro abgebucht werden. Das Transaktionsende ist das Eingehen des Geldes auf Konto B. Im Snapshot wurde dann lediglich das Geld von Konto A abgebucht, aber nicht auf Konto B überwiesen.

Um diesen Nachteil zu beheben und Anwendungskonsistenz zu gewährleisten gibt es verschiedene Möglichkeiten. Zum einen ist es möglich, der Anwendung mitzuteilen, dass gleich ein Snapshot gemacht wird. Die Anwendung muss dann dafür sorgen, dass keine halben Transaktionen auf die Platte geschrieben werden. Es darf in dieser Zeit keine neue Transaktion begonnen werden und die laufende Transaktion muss abgeschlossen werden, bevor pausiert wird. Eine weitere Möglichkeit den Nachteil zu beheben ist, die unvollständige Transaktion (gekennzeichnet durch das fehlende Commit) komplett zu verwerfen. Beispielsweise würden die 100 Euro wieder auf das Konto A zurückgebucht werden.

```
matthias@matthias-VirtualBox:~$ python3 writer.py
Programm gestartet. Schreiben des Anfangs
Anfang geschrieben. Warten für 20 Sekunden. Jetzt Snapshot machen
Ende geschrieben. Vorgang abgeschlossen.
matthias@matthias-VirtualBox:~$ cd /pool1/daten_x/.zfs/snapshot/20260307_113443
matthias@matthias-VirtualBox:/pool1/daten_x/.zfs/snapshot/20260307_113443$ cat transaction.txt
--- TRANSAKTION START ---
matthias@matthias-VirtualBox:/pool1/daten_x/.zfs/snapshot/20260307_113443$
```



```
matthias@matthias-VirtualBox:~$ python3 backup.py
--- Starte Backup: 20260307_113443 ---
Erstelle Snapshot: pool1/daten_x@20260307_113443
[sudo] Passwort für matthias:
Sende Snapshot in Datei: /home/matthias/zfs_backups/backup_20260307_113443.zfs
Retention: Lösche altes Backup backup_20260307_112017.zfs
--- Backup abgeschlossen ---
```

Aufgabe 3: Datensicherheit dank raidz und raidz2

Ziel dieser Aufgabe ist es zu zeigen, dass der Ausfall einer Platte keinen unmittelbaren Einfluss auf das Dateisystem hat und dass alle Anwendungen, die zum Zeitpunkt des Ausfalls auf Dateien im zugehörigen ZFS-Dataset lesend und schreibend zugreifen, ohne Unterbrechung weiterarbeiten können. Hierzu wird die raidz-Technik verwendet. Bei raidz1 werden Daten und Paritätsinformationen so über mindestens drei Festplatten verteilt, dass beim Ausfall einer beliebigen Platte keine Daten verloren gehen. ZFS ist in der Lage, die fehlenden Datenblöcke der ausgefallenen Platte in Echtzeit aus den verbliebenen Daten- und Paritätsblöcken der anderen Festplatten zu berechnen.

Für das Experiment wurde ein neuer Speicherpool pool2 erstellt. Da raidz1 mindestens drei Festplatten benötigt, wurden in der virtuellen Maschine die drei Platten sdb, sdc, und sdd zusammengeschaltet (sudo zpool create -f pool2 raidz sdb sdc sdd). Um die Auswirkungen eines Festplattenausfalls auf eine laufende

Anwendung zu prüfen, wurde ein Testscript implementiert. Dieses Script schreibt in einer Endlosschleife eine neue Zeile in einer Log-Datei auf dem raidz-dataset. Anschließend wird die geschriebene Zeile sofort zur Verifizierung ausgelesen und im Terminal wird eine Ausgabe geprintet. Während dieses Schreibe-Lese-Zyklus wird der Ausfall einer Festplatte simuliert. Dazu wurde die Festplatte sdb in den Offline-Zustand versetzt. (sudo zpool offline pool2 sdb).

Trotz des simulierten Hardwaredefekts lief das Pythonscript ohne Fehlermeldung oder Verzögerung weiter. Eine Abfrage des Pool-Status zeigt, dass ZFS den Defekt erkannt hat. Der Poolstatus hat von ONLINE zu DEGRADED gewechselt. Das Gerät sdb wird als OFFLINE angezeigt, während der Pool als Ganzes weiterhin funktioniert.

```
Schreib/Lese-Test 134 erfolgreich: Zeile 134: System läuft stabil
Schreib/Lese-Test 135 erfolgreich: Zeile 135: System läuft stabil
Schreib/Lese-Test 136 erfolgreich: Zeile 136: System läuft stabil
Schreib/Lese-Test 137 erfolgreich: Zeile 137: System läuft stabil
Schreib/Lese-Test 138 erfolgreich: Zeile 138: System läuft stabil
Schreib/Lese-Test 139 erfolgreich: Zeile 139: System läuft stabil
Schreib/Lese-Test 140 erfolgreich: Zeile 140: System läuft stabil
Schreib/Lese-Test 141 erfolgreich: Zeile 141: System läuft stabil
Schreib/Lese-Test 142 erfolgreich: Zeile 142: System läuft stabil
^CTraceback (most recent call last):
  File "zfs_test.py", line 10, in <module>
    zpool_status = subprocess.run(["zpool", "status", "pool2"], capture_output=True)
  File "/usr/lib/python3.10/subprocess.py", line 526, in run
    raise CalledProcessError(retcode, process.args)
subprocess.CalledProcessError: Command '['zpool', 'status', 'pool2']' returned non-zero exit code 1.
```

The screenshot shows a terminal window titled "matthias@matthias-VirtualBox: ~". The user has entered the command `[sudo] Passwort für matthias:` and then `matthias@matthias-VirtualBox:~$ zpool status pool2`. The output indicates that the pool is in a DEGRADED state because one or more devices have been taken offline by the administrator. It lists the devices and their states: `pool2` is DEGRADED, `raidz1-0` is DEGRADED, `sdb` is OFFLINE, `sdc` is ONLINE, and `sdd` is ONLINE. The READ, WRITE, and CKSUM values are all 0 for the degraded components.

```
pool: pool2
state: DEGRADED
status: One or more devices has been taken offline by the administrator.
        Sufficient replicas exist for the pool to continue functioning in a
        degraded state.
action: Online the device using 'zpool online' or replace the device with
        'zpool replace'.
config:

NAME      STATE     READ WRITE CKSUM
pool2     DEGRADED   0     0     0
  raidz1-0 DEGRADED   0     0     0
    sdb     OFFLINE    0     0     0
    sdc     ONLINE     0     0     0
    sdd     ONLINE     0     0     0
```

Aufgabe 4: Zugriffsperformanz

Um die Performanzunterschiede zwischen ZFS und dem Standard-Linux-Dateisystem ext4 zu ermitteln, wurden beide Systeme auf identischen virtuellen Festplatten (5 GB) innerhalb der virtuellen Maschine installiert. Das ext4-Dateisystem wurde auf dem Datenträger /dev/sdb erstellt und unter /mnt/ext4_test eingebunden. ZFS wurde als Referenz auf dem identischen Datenträger /dev/sdc konfiguriert. Für die Messung wurde fio (Flexible I/O Tester) verwendet. Es wurden 2 Szenarien getestet.

Szenario 1: Sequenzielles Schreiben (1MB Blöcke, 1GB Dateigröße)

Dieses Szenario verwendet große Datenblöcke und simuliert das Schreiben einer großen, zusammenhängenden Datei.

Ext4: `sudo fio --name=seqwrite_ext4 --rw=write --bs=1M --size=1G --end_fsync=1 --directory=/mnt/ext4_test/bench`

```
Run status group 0 (all jobs):
  WRITE: bw=208MiB/s (218MB/s), 208MiB/s-208MiB/s (218MB/s-218MB/s), io=1024MiB (1074
B), run=4926-4926msec
```

ZFS: `sudo fio --name=seqwrite_zfs --rw=write --bs=1M --size=1G --end_fsync=1 --directory=/pool_bench/test/bench`

```
Run status group 0 (all jobs):
  WRITE: bw=392MiB/s (411MB/s), 392MiB/s-392MiB/s (411MB/s-411MB/s), io=1024MiB (1074M
B), run=2610-2610msec
```

Beim sequenziellen Schreiben großer Blöcke ist ZFS (411MB/s) fast doppelt so schnell wie ext4 (218MB/s). In diesem Szenario erzeugt fio leicht komprimierbare Testmuster. Da ZFS über eine transparente Datenkompression verfügt, werden die Daten im RAM komprimiert. ZFS muss demnach physisch weniger Daten auf die eigentliche Platte schreiben.

Szenario 2: Zufälliges Schreiben (4KB Blöcke, 500MB Dateigröße)

Dieses Szenario verwendet kleinere Datenblöcke und simuliert zufällige Schreibzugriffe auf dem gesamten Datenträger.

Ext4: `sudo fio --name=randwrite_ext4 --rw=randwrite --bs=4k --size=500M --end_fsync=1 --directory=/mnt/ext4_test/bench`

```
Run status group 0 (all jobs):
  WRITE: bw=141MiB/s (148MB/s), 141MiB/s-141MiB/s (148MB/s-148MB/s), io=500MiB (524MB)
, run=3535-3535msec
```

ZFS: `sudo fio --name=randwrite_zfs --rw=randwrite --bs=4k --size=500M --end_fsync=1 --directory=/pool_bench/test/bench`

```
Run status group 0 (all jobs):
  WRITE: bw=45.3MiB/s (47.5MB/s), 45.3MiB/s-45.3MiB/s (47.5MB/s-47.5MB/s), io=500MiB (
524MB), run=11031-11031msec
```

Bei zufälligen, kleinen Schreibzugriffen verändert sich das Ergebnis. Ext4 (148MB/s) ist nun etwa dreimal so schnell wie ZFS (47MB/s). Während ext4 bestehende Datenblöcke auf der Festplatte direkt überschreiben, schreibt ZFS geänderte Blöcke immer an eine neue, freie Stelle. Jeder Schreibzugriff erfordert, dass ZFS einen neuen freien Block sucht, Prüfsummen berechnet und Metadaten aktualisiert.